BÁO CÁO THỰC HÀNH
LAB 3

ỨNG DỤNG FREERTOS TRÊN ESP32
GIÁM SÁT MÔI TRƯỜNG VƯỜN CÂY CÔNG NGHIỆP
QUA MQTT VÀ WEB DASHBOARD

(Đề 2 – Cảm biến DHT22, máy phun sương điều khiển PWM)

Sinh viên:	Nguyễn Tiến Hiếu
MSSV:	231432233
Lớp:	Cử nhân kĩ thuật điện tử viễn thông K64
Đề thực hiện:	Lab 3 – Đề 2
Mô phỏng:	Wokwi – ESP32 DevKitC V4
Link Wokwi:	https://wokwi.com/projects/479992109740419073

1. YÊU CẦU CỦA ĐỀ BÀI

1.1. Yêu cầu chung về thực hiện và trình bày (Lab 3)

Theo tài liệu Lab 3, báo cáo phải có đủ các nội dung sau:

- Giới thiệu ưu điểm của RTOS, ưu điểm và nhược điểm của FreeRTOS.
- Giới thiệu các cơ chế giao tiếp giữa các Task như cờ hiệu (Semaphore) và hàng đợi (Queue), nêu ưu điểm và ứng dụng.
- Trình bày các bước thực hiện bài Lab, gồm:
  - Sơ đồ mạch trong Wokwi, các chân kết nối của ESP32 với cảm biến và thiết bị chấp hành.
  - Triển khai FreeRTOS trên ESP32 và bảng dữ liệu cho các Task.
  - Tạo giao diện website/app để hiển thị dữ liệu và điều khiển.
  - Code và giải thích code quá trình tạo kết nối, cập nhật dữ liệu và điều khiển.
- Trình bày kết quả đạt được bằng hình ảnh (hoặc video nếu có); gửi link project Wokwi có ghi rõ họ tên – lớp.

1.2. Yêu cầu riêng của Đề 2

Đề 2: Thiết kế mạch giám sát dữ liệu môi trường vườn trồng cây công nghiệp (cà phê, hồ tiêu). ESP32 đọc nhiệt độ, độ ẩm từ cảm biến DHT22, hiển thị qua giao tiếp Serial, đồng thời gửi dữ liệu lên MQTT Broker qua Wi-Fi và hiển thị trên Web/App. Các yêu cầu cụ thể:

- Vẽ mạch kết nối trong Wokwi.
- Giao diện Web/App lấy dữ liệu từ Broker: 2 thẻ HTML hiển thị nhiệt độ và độ ẩm dạng số; 2 biểu đồ cập nhật nhiệt độ và độ ẩm theo thời gian thực (tùy chọn); 2 nút button điều khiển Máy bơm và Quạt (chế độ manual).
- Vườn cần độ ẩm ổn định **40% – 60%**; xây dựng thuật toán phù hợp, thêm **máy phun sương** bật khi độ ẩm dưới ngưỡng.
- Xây dựng hệ RTOS gồm 3 Task:
  - **Task truyền data** (Core 0): kết nối Wi-Fi; nếu thành công thì đọc dữ liệu từ hàng đợi data Queue và gửi lên MQTT Broker.
  - **Task đọc data** (Core 1): đọc DHT22, hiển thị qua Serial, gửi dữ liệu đầy đủ vào data Queue và gửi độ ẩm vào data Alert.
  - **Task cảnh báo** (Core 1): đọc data Alert và điều khiển máy phun sương theo thuật toán ở Bảng 1.

Bảng 1. Thuật toán điều khiển máy phun sương theo đề

Độ ẩm	Công suất máy phun	Thời gian
Trên 40%	Không Phun	0 giây
35% – 40%	25%	10 giây
30% – 35%	50%	20 giây
Dưới 30%	100%	30 giây

1.3. Phân tích yêu cầu và mục tiêu cần đạt

Từ đề bài, mục tiêu của bài Lab được xác định như sau:

- Vận dụng FreeRTOS trên ESP32 hai nhân: chia chương trình thành các Task độc lập, **ghim đúng Core** theo đề (truyền dữ liệu ở Core 0, đọc và cảnh báo ở Core 1).
- Dùng **Queue** để trao đổi dữ liệu giữa các Task thay cho biến toàn cục; tách riêng hai luồng dữ liệu data Queue (đủ nhiệt độ + độ ẩm) và data Alert (chỉ độ ẩm).
- Truyền thông qua **MQTT**: ESP32 publish dữ liệu cảm biến và subscribe lệnh điều khiển; Web Dashboard làm việc ngược lại.
- Cài đặt thuật toán phun sương có **công suất (PWM)** và **thời gian chạy** theo từng dải độ ẩm, không dùng delay() chặn để các Task khác vẫn hoạt động.

Bảng 2 đối chiếu từng yêu cầu của đề với hạng mục đã thực hiện trong báo cáo này.

Bảng 2. Đối chiếu yêu cầu của đề với kết quả thực hiện

Yêu cầu của đề	Cách đáp ứng	Mục
Vẽ mạch trong Wokwi	ESP32 DevKitC V4 + DHT22 + 3 LED (phun sương, bơm, quạt)	3.2
Đọc DHT22, hiển thị Serial	TaskReadData in nhiệt độ, độ ẩm mỗi 2 giây	3.6, 4
Gửi dữ liệu lên MQTT Broker	TaskMQTT publish JSON lên topic lab3/de2/sensor	3.5, 3.6
2 thẻ số + 2 biểu đồ	Web Dashboard: thẻ nhiệt độ, độ ẩm và 2 biểu đồ SVG 20 mẫu gần nhất	3.5
2 nút điều khiển bơm, quạt	Nút BẬT/TẮT publish lệnh ON/OFF; ESP32 xử lý trong mqttCallback	3.5, 4
Độ ẩm ổn định 40% – 60%	Dưới 40% thì phun theo 3 mức; từ 40% trở lên tắt phun; web cảnh báo khi vượt 60%	3.4

Yêu cầu của đề	Cách đáp ứng	Mục
Task truyền data – Core 0	TaskMQTT ghim Core 0	3.3
Task đọc data – Core 1	TaskReadData ghim Core 1, gửi 2 Queue	3.3
Task cảnh báo – Core 1	TaskAlert ghim Core 1, điều khiển PWM và đếm ngược	3.3, 3.6
Thuật toán 25% / 50% / 100%	getHumidityLevel() + switch(level) trong TaskAlert	3.4

2. CƠ SỞ LÝ THUYẾT

2.1. Ưu điểm của RTOS

RTOS (Real-Time Operating System – hệ điều hành thời gian thực) là hệ điều hành nhỏ gọn cho vi điều khiển, cho phép nhiều tác vụ (Task) chạy song song về mặt logic và đáp ứng sự kiện trong thời gian xác định. So với cách viết một vòng lặp loop() tuần tự, RTOS có các ưu điểm chính:

- **Đa nhiệm:** mỗi chức năng (đọc cảm biến, truyền mạng, điều khiển thiết bị) là một Task riêng, chạy độc lập mà không chờ nhau.
- **Tính thời gian thực, có tính xác định:** bộ lập lịch (scheduler) cấp CPU theo mức ưu tiên, nên tác vụ quan trọng được phục vụ trước.
- **Quản lý thời gian hiệu quả:** vTaskDelay() đưa Task vào trạng thái chờ và nhường CPU cho Task khác, khác với delay() làm treo cả chương trình.
- **Dễ bảo trì và mở rộng:** chương trình chia thành các module nhỏ, thêm hoặc sửa một chức năng ít ảnh hưởng phần còn lại.
- **Cơ chế đồng bộ và giao tiếp sẵn có:** Queue, Semaphore, Mutex, Event Group giúp các Task trao đổi dữ liệu an toàn.
- **Tận dụng vi điều khiển đa nhân** (như ESP32) bằng cách phân bổ Task lên từng Core.

2.2. Ưu điểm và nhược điểm của FreeRTOS

FreeRTOS là hạt nhân RTOS mã nguồn mở, được tích hợp sẵn trong framework của ESP32 (ESP-IDF và Arduino-ESP32).

Bảng 3. Ưu điểm và nhược điểm của FreeRTOS

Ưu điểm	Nhược điểm
Mã nguồn mở (giấy phép MIT), miễn phí, sử dụng được cho cả sản phẩm thương mại.	Chỉ là kernel: không có sẵn hệ thống tệp, ngăn xếp mạng, driver; phải dùng thêm thư viện.
Nhỏ gọn, chiếm ít RAM/Flash, chạy được trên nhiều dòng vi điều khiển.	Mỗi Task cần tự khai báo kích thước stack; chọn sai dễ gây tràn stack.
Tích hợp sẵn trên ESP32, hỗ trợ ghim Task vào Core (xTaskCreatePinnedToCore).	Lập trình viên phải tự xử lý đồng bộ; thiết kế kém dễ gặp race condition, deadlock, đảo ngược ưu tiên.
Đủ công cụ: Task, Queue, Semaphore, Mutex, Software Timer, Event Group.	Gỡ lỗi khó hơn chương trình tuần tự vì lỗi phụ thuộc thời điểm chạy.
Tài liệu phong phú, cộng đồng lớn, nhiều ví dụ.	Bản chuẩn không có bảo vệ bộ nhớ giữa các Task; mỗi Task thêm tiêu tốn RAM cho stack.

2.3. Cơ chế giao tiếp giữa các Task: Semaphore và Queue

2.3.1. Semaphore (cờ hiệu)

Semaphore là cơ chế dùng để **báo hiệu và đồng bộ**, không mang dữ liệu. Có ba dạng thường dùng:

- **Binary Semaphore:** chỉ có hai trạng thái (có/không), dùng để báo một sự kiện đã xảy ra, ví dụ ngắt (ISR) báo cho Task xử lý.
- **Counting Semaphore:** đếm số lượng sự kiện hoặc số tài nguyên còn khả dụng.
- **Mutex:** loại đặc biệt để bảo vệ tài nguyên dùng chung (Serial, bus I2C/SPI), có cơ chế kế thừa ưu tiên (priority inheritance) để hạn chế đảo ngược ưu tiên.

Ưu điểm: đồng bộ giữa các Task, tránh hai Task cùng truy cập một tài nguyên, chi phí thấp. **Ứng dụng:** báo hiệu từ ngắt (nhấn nút, đo xung), bảo vệ cổng Serial khi nhiều Task cùng in, quản lý số kết nối hoặc số bộ đệm còn trống.

2.3.2. Queue (hàng đợi)

Queue là bộ đệm theo cơ chế FIFO để **truyền dữ liệu** giữa các Task (hoặc giữa ngắt và Task). Dữ liệu được sao chép vào Queue theo giá trị, có thể là kiểu cơ bản hoặc cấu trúc struct. Task gửi và Task nhận có thể chờ có giới hạn thời gian (timeout) khi Queue đầy hoặc rỗng.

- **Ưu điểm:** an toàn khi nhiều Task/Core cùng truy cập (không cần tự khóa); tách rời bên sản xuất và bên tiêu thụ dữ liệu; không cần biến toàn cục; Task nhận tự ngủ khi chưa có dữ liệu nên tiết kiệm CPU.
- **Ứng dụng:** chuyển dữ liệu cảm biến từ Task đọc sang Task xử lý hoặc Task truyền mạng, gửi lệnh điều khiển, gom nhật ký (log) từ nhiều Task.

Bảng 4. So sánh Semaphore và Queue

Tiêu chí	Semaphore	Queue
Mục đích	Báo hiệu, đồng bộ, bảo vệ tài nguyên	Truyền dữ liệu giữa các Task
Mang dữ liệu	Không	Có (giá trị hoặc struct)
Ví dụ trong bài	Có thể dùng Mutex bảo vệ Serial	dataQueue, dataAlert

Trong bài Lab này các Task cần trao đổi **giá trị đo** (nhiệt độ, độ ẩm) nên Queue là lựa chọn phù hợp. Cả hai Queue đều có dung lượng 1 phần tử và được ghi bằng xQueueOverwrite(), nên Queue luôn giữ mẫu **mới nhất** và không bị đầy.

2.4. Giao thức MQTT

MQTT là giao thức nhẹ chạy trên TCP theo mô hình **publish/subscribe**: các thiết bị không nối trực tiếp với nhau mà thông qua **Broker**. Bên gửi publish bản tin lên một **topic**, mọi bên đã subscribe topic đó đều nhận được. Bài Lab dùng Broker công cộng broker.hivemq.com: ESP32 kết nối qua TCP cổng 1883, còn Web Dashboard kết nối qua WebSocket bảo mật (wss://broker.hivemq.com:8884/mqtt) vì trình duyệt không mở được kết nối TCP thuần. Các bản tin dùng QoS 0 (gửi tối đa một lần), phù hợp dữ liệu cảm biến được làm mới liên tục.

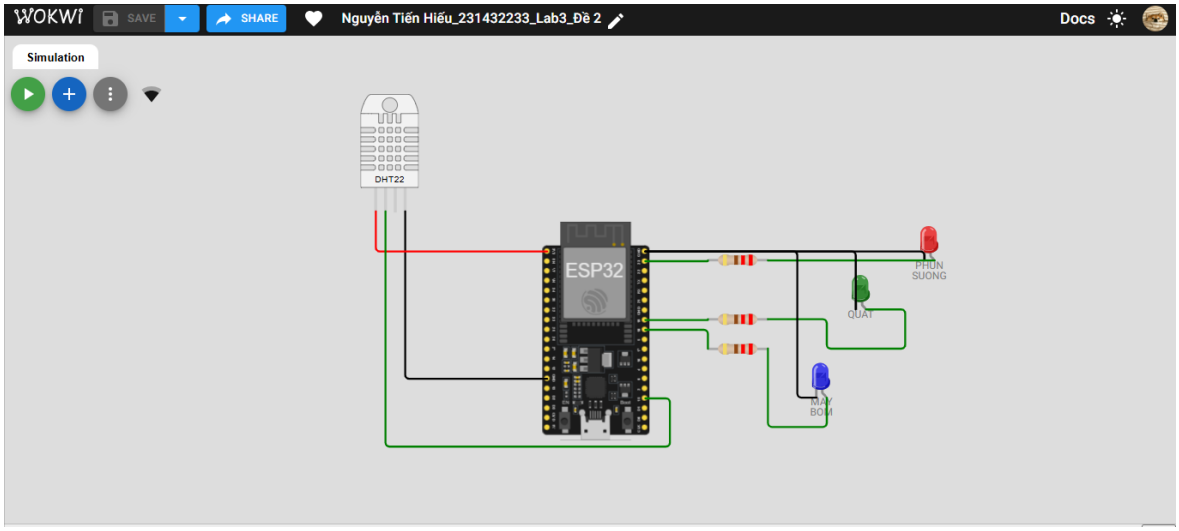
3. THIẾT KẾ VÀ THỰC HIỆN

3.1. Công cụ và thư viện

- **Mô phỏng:** Wokwi, board ESP32 DevKitC V4, cảm biến DHT22, 3 LED đại diện cho máy phun sương, máy bơm và quạt.
- **Thư viện phía ESP32:** WiFi.h (kết nối Wi-Fi), PubSubClient (MQTT), DHT (đọc DHT22); FreeRTOS có sẵn trong core Arduino-ESP32.
- **Web Dashboard:** một tệp HTML (Link_web_lab3_đề_2.html) dùng thư viện mqtt.js 5.10.3 nạp từ CDN; biểu đồ vẽ bằng SVG thuần, không cần thư viện biểu đồ.
- **Mạng:** Wi-Fi ảo Wokwi-GUEST (không mật khẩu) và Broker công cộng HiveMQ.

3.2. Sơ đồ mạch trong Wokwi

Sơ đồ mạch được vẽ trên Wokwi (Hình 1). Cảm biến DHT22 nối vào GPIO15. Ba LED được nối qua điện trở hạn dòng vào ba chân GPIO đầu ra, mỗi LED đại diện cho một cơ cấu chấp hành: LED đỏ là máy phun sương (điều khiển PWM), LED xanh dương là máy bơm và LED xanh lá là quạt (điều khiển BẬT/TẮT). Cathode các LED nối GND.



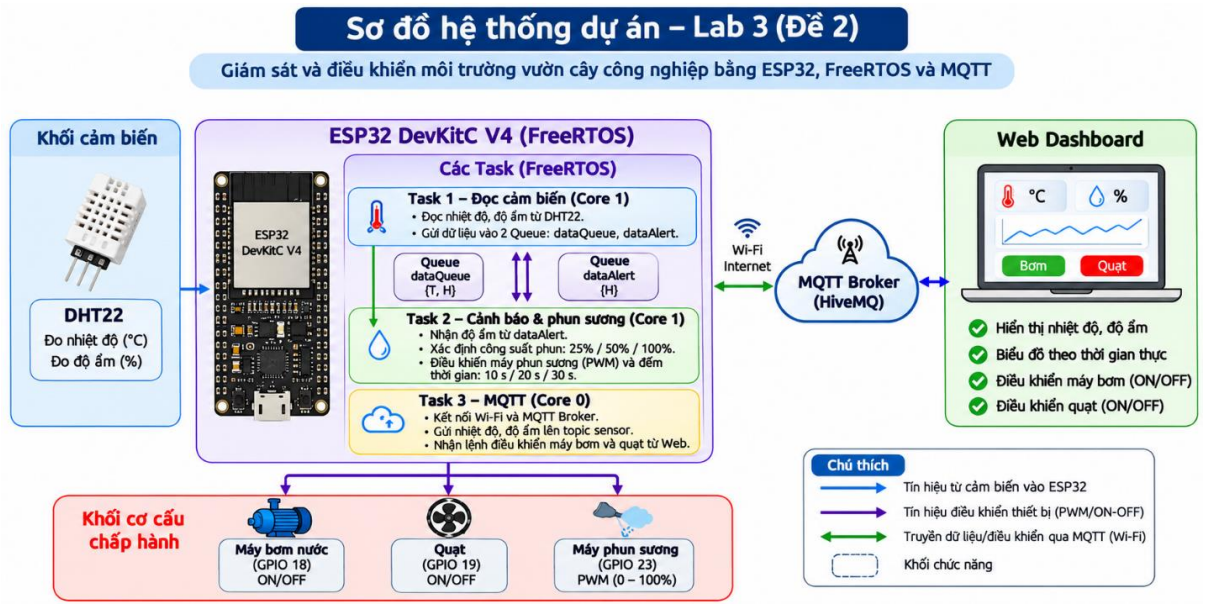
Hình 1. *Sơ đồ mạch ESP32 DevKitC V4, DHT22 và ba LED trên Wokwi*

Bảng 5. Bảng chân kết nối

Thiết bị	Chân thiết bị	Chân ESP32	Ghi chú
DHT22	VCC	3V3	Nguồn 3,3 V
DHT22	SDA (DATA)	GPIO15	Đọc nhiệt độ, độ ẩm
DHT22	GND	GND	
Máy phun sương (LED đỏ)	Anode (qua điện trở)	GPIO23	Xuất PWM 0 – 255
Máy bơm (LED xanh dương)	Anode (qua điện trở)	GPIO18	BẬT/TẮT từ web
Quạt (LED xanh lá)	Anode (qua điện trở)	GPIO19	BẬT/TẮT từ web

3.3. Triển khai FreeRTOS trên ESP32

Hệ thống gồm 3 Task và 2 Queue (Hình 2). TaskMQTT chạy ở Core 0 vì khối Wi-Fi của ESP32 thường hoạt động trên Core 0; hai Task còn lại chạy ở Core 1 đúng theo yêu cầu của đề.



Hình 2. *Sơ đồ hệ thống dự án Lab 3 (Đề 2)*

Bảng 6. Bảng dữ liệu và tổ chức các Task

Task	Core	Ưu tiên	Stack	Chức năng / chu kỳ
TaskReadData	1	2	4096	Đọc DHT22, in Serial, gửi hai Queue; chu kỳ 2000 ms
TaskAlert	1	2	4096	Nhận độ ẩm, điều khiển PWM, đếm ngược không chặn; chờ Queue 50 ms rồi nghỉ 50 ms
TaskMQTT	0	1	6144	Kết nối Wi-Fi/MQTT, publish dữ liệu, nhận lệnh bơm/quạt; nghỉ 20 ms khi đã online

Bảng 7. Bảng dữ liệu các Queue

Queue	Kiểu dữ liệu	Dung lượng	Gửi → Nhận
dataQueue	SensorData { float temperature; float humidity; }	1 phần tử	TaskReadData → TaskMQTT
dataAlert	float humidity	1 phần tử	TaskReadData → TaskAlert

Nhận xét về ưu tiên: TaskReadData và TaskAlert cùng ưu tiên 2 trên Core 1 nên chia sẻ CPU luân phiên; cả hai đều nhường CPU bằng vTaskDelay() hoặc chờ Queue có timeout. TaskMQTT nằm ở Core 0 nên không tranh CPU trực tiếp với hai Task kia. Chu kỳ của TaskAlert chỉ là gần đúng vì phụ thuộc thời gian chờ nhận dữ liệu.

3.4. Thuật toán điều khiển máy phun sương

Dải độ ẩm mục tiêu của vườn là 40% – 60%. Khi độ ẩm xuống dưới 40%, máy phun sương được bật theo ba mức; từ 40% trở lên máy tắt. Hàm getHumidityLevel() quy độ ẩm về mức 0 – 3, từ đó TaskAlert chọn công suất và thời gian (Bảng 8). Giá trị PWM được tính bằng map(percent, 0, 100, 0, 255).

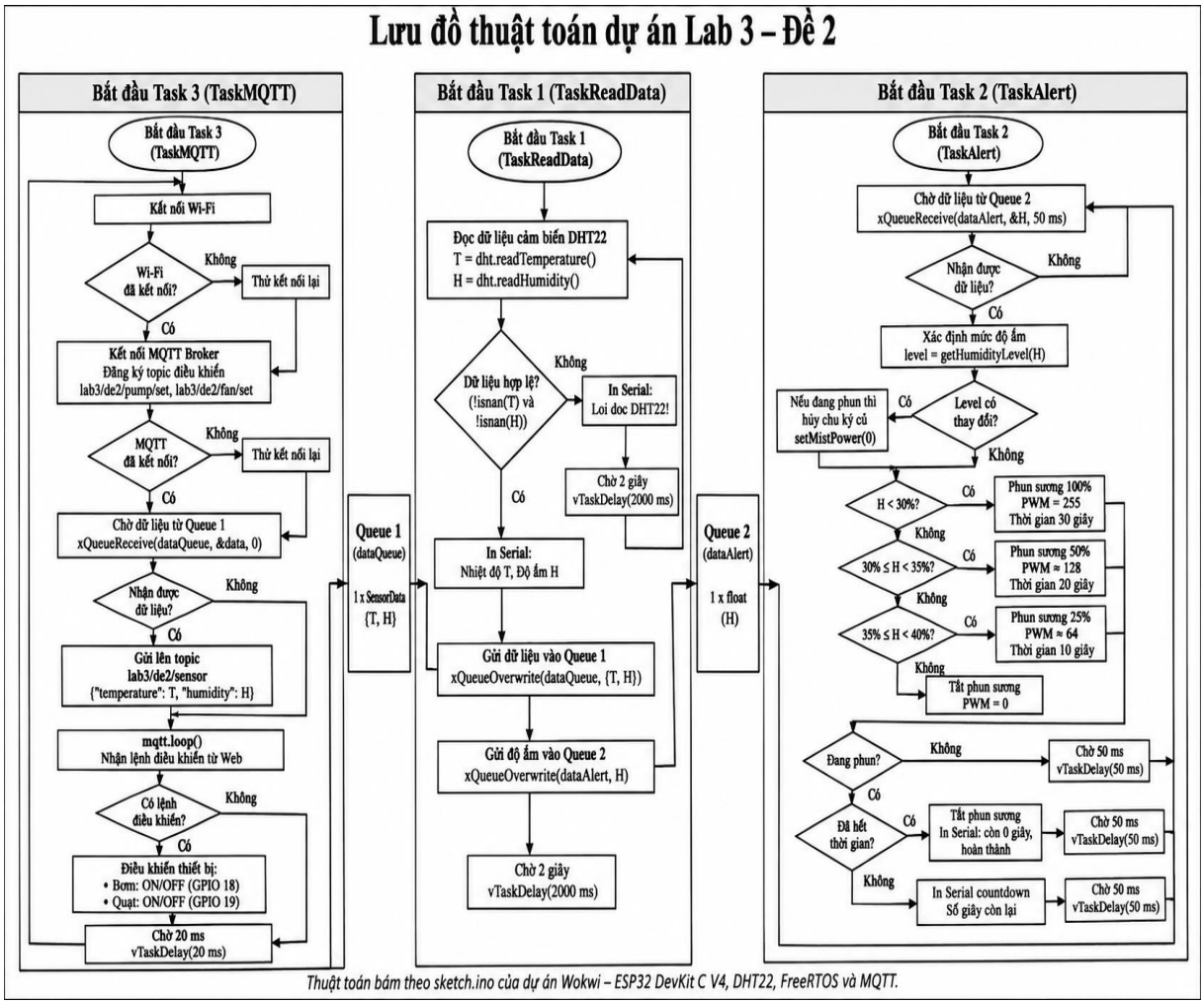
Bảng 8. Điều kiện điều khiển máy phun sương theo code

Độ ẩm H (%)	Mức (level)	Công suất	PWM	Thời gian
H ≥ 40	0	0%	0	Tắt
35 ≤ H < 40	1	25%	63 / 255	10 giây
30 ≤ H < 35	2	50%	127 / 255	20 giây
H < 30	3	100%	255 / 255	30 giây

Các quy tắc xử lý cụ thể trong chương trình:

- **Biên dải:** dải dưới dùng toán tử <, nên đúng 30%, 35% và 40% thuộc dải phía trên.
- **Chỉ khởi động chu kỳ khi đổi dải:** nếu độ ẩm vẫn ở cùng một dải thì không bắt đầu chu kỳ phun mới; khi chuyển dải, chu kỳ cũ bị hủy và chu kỳ mới bắt đầu.
- **Đếm ngược không chặn:** thời gian phun được đo bằng millis() trong vòng lặp 50 ms, nên Task vẫn nhận được dữ liệu mới và không làm treo các Task khác.
- **Độ ẩm trên 60%:** đề không yêu cầu thiết bị nào xử lý nên chương trình không phun; Web hiển thị nhận xét “Độ ẩm trên khoảng mục tiêu”.

Lưu đồ thuật toán



Hình 3. *Lưu đồ thuật toán dự án Lab 3 – Đề 2 (TaskMQTT, TaskReadData, TaskAlert)*

3.5. Giao diện Web Dashboard

Giao diện là một tệp HTML đơn, mở trực tiếp trên trình duyệt. Trang tự kết nối tới Broker HiveMQ bằng mqtt.js qua WebSocket bảo mật và subscribe topic dữ liệu cảm biến. Các thành phần chính:

- **Thẻ số:** thẻ Nhiệt độ (id="temp"), thẻ Độ ẩm (id="hum") kèm nhận xét theo dải mục tiêu 40% – 60%, và thẻ Máy phun sương hiển thị mức công suất suy ra từ độ ẩm.
- **Hai biểu đồ** nhiệt độ và độ ẩm vẽ bằng SVG, lưu 20 mẫu gần nhất và cập nhật mỗi khi nhận bản tin mới.
- **Điều khiển thủ công:** hai khối Máy bơm và Quạt, mỗi khối có nút BẬT và TẮT; các nút bị khóa cho đến khi MQTT kết nối thành công.
- **Nhật ký hoạt động:** ghi lại sự kiện kết nối, dữ liệu nhận được và lệnh đã gửi.
- **Kiểm tra dữ liệu:** bản tin không phải JSON, hoặc giá trị ngoài khoảng hợp lý (độ ẩm 0 – 100%, nhiệt độ −40 đến 85 °C), bị bỏ qua; nếu quá 12 giây không có dữ liệu mới, trang gắn nhãn “dữ liệu cũ”.

Bảng 9. Các topic MQTT sử dụng

Topic	Hướng	Nội dung bản tin
lab3/de2/sensor	ESP32 → Web	JSON {"temperature": 28.0, "humidity": 71.5}
lab3/de2/pump/set	Web → ESP32	Chuỗi ON hoặc OFF (chấp nhận cả 1 và 0)
lab3/de2/fan/set	Web → ESP32	Chuỗi ON hoặc OFF (chấp nhận cả 1 và 0)

Đoạn mã chính của Web: cấu hình MQTT, nhận dữ liệu cảm biến và gửi lệnh điều khiển.

```
const CONFIG = Object.freeze({
  broker: 'wss://broker.hivemq.com:8884/mqtt',
  sensor: 'lab3/de2/sensor',
  pump:   'lab3/de2/pump/set',
  fan:    'lab3/de2/fan/set'
});

// Nhan du lieu cam bien tu Broker
client.on('message', (topic, payload) => {
  const data = JSON.parse(payload.toString());
  if (topic === CONFIG.sensor) {
    const t = Number(data.temperature), h = Number(data.humidity);
    if (record(t, h)) log('DHT22: ' + t.toFixed(1) + ' C | ' + h.toFixed(1) + ' %');
  }
```

```
});

// Gui lenh dieu khien bom/quat
function publishControl(device, action) {
  const topic = device === 'pump' ? CONFIG.pump : CONFIG.fan;
  state.client.publish(topic, action, { qos: 0, retain: false });
}
```

Giải thích: hàm record() kiểm tra tính hợp lệ, lưu mẫu vào lịch sử 20 phần tử, cập nhật hai thẻ số, mức phun sương và vẽ lại hai biểu đồ. Hàm publishControl() publish chuỗi ON hoặc OFF lên topic của thiết bị; ESP32 nhận lệnh trong mqttCallback().

3.6. Code ESP32 và giải thích

Toàn bộ mã nguồn nằm ở Phụ lục A. Phần này giải thích các khối chính theo thứ tự chạy của chương trình.

3.6.1. Khai báo chân, cấu hình Wi-Fi/MQTT và Queue

```
#define DHT_PIN 15
#define PUMP_PIN 18
#define FAN_PIN 19
#define MIST_PIN 23

const char* WIFI_SSID = "Wokwi-GUEST";
const char* MQTT_SERVER = "broker.hivemq.com";
const char* TOPIC_SENSOR = "lab3/de2/sensor";
const char* TOPIC_PUMP = "lab3/de2/pump/set";
const char* TOPIC_FAN = "lab3/de2/fan/set";

struct SensorData { float temperature; float humidity; };
QueueHandle_t dataQueue;
QueueHandle_t dataAlert;
```

Các chân GPIO khớp với sơ đồ mạch ở Bảng 5. Cấu trúc SensorData gộp nhiệt độ và độ ẩm để gửi qua dataQueue trong một lần; dataAlert chỉ chứa độ ẩm vì Task cảnh báo không cần nhiệt độ.

3.6.2. Hàm xác định mức độ ẩm và điều khiển PWM

```
int getHumidityLevel(float humidity) {
  if (humidity < 30.0) return 3;
  if (humidity < 35.0) return 2;
  if (humidity < 40.0) return 1;
  return 0;
}

void setMistPower(int percent) {
  percent = constrain(percent, 0, 100);
  int pwm = map(percent, 0, 100, 0, 255);
  analogWrite(MIST_PIN, pwm);
```

```
}
```

getHumidityLevel() kiểm tra từ ngưỡng thấp nhất lên nên cho đúng mức theo Bảng 8. setMistPower() giới hạn công suất trong 0 – 100%, đổi sang giá trị PWM 0 – 255 và xuất ra GPIO23 bằng analogWrite().

3.6.3. Nhận lệnh điều khiển thủ công (mqttCallback)

```
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (unsigned int i = 0; i < length; i++) message += (char)payload[i];
  message.trim();
  message.toUpperCase();

  if (String(topic) == TOPIC_PUMP) {
    if (message == "ON" || message == "1") digitalWrite(PUMP_PIN, HIGH);
    else if (message == "OFF" || message == "0") digitalWrite(PUMP_PIN, LOW);
  }
  // Quat xu ly tuong tu voi TOPIC_FAN va FAN_PIN
}
```

Hàm được PubSubClient gọi mỗi khi có bản tin trên topic đã subscribe. Nội dung được ghép thành chuỗi, cắt khoảng trắng, đổi sang chữ hoa rồi so sánh với ON/OFF, nên web gửi chữ thường hay hoa đều được. Tùy topic mà bật hoặc tắt chân GPIO của máy bơm hoặc quạt.

3.6.4. TaskReadData – đọc cảm biến (Core 1)

```
void TaskReadData(void* pvParameters) {
  SensorData data;
  while (true) {
    data.temperature = dht.readTemperature();
    data.humidity = dht.readHumidity();

    if (isnan(data.temperature) || isnan(data.humidity)) {
      Serial.println("[DHT22] Loi doc cam bien!");
    } else {
      Serial.printf("Nhiet do : %.1f C\n", data.temperature);
      Serial.printf("Do am    : %.1f %%\n", data.humidity);
      xQueueOverwrite(dataQueue, &data);
      xQueueOverwrite(dataAlert, &data.humidity);
    }
    vTaskDelay(pdMS_TO_TICKS(2000));
  }
}
```

Mỗi 2 giây Task đọc DHT22; nếu giá trị hợp lệ thì in ra Serial rồi ghi vào cả hai Queue. xQueueOverwrite() ghi đè mẫu cũ nên Queue 1 phần tử không bao giờ đầy, và các Task nhận luôn xử lý số liệu mới nhất. vTaskDelay() nhường CPU trong thời gian chờ.

3.6.5. TaskAlert – cảnh báo và điều khiển máy phun (Core 1)

```
if (xQueueReceive(dataAlert, &humidity, pdMS_TO_TICKS(50)) == pdPASS) {
  int level = getHumidityLevel(humidity);
  if (level != lastLevel) {          // chi xu ly khi doi dai do am
    setMistPower(0);                 // huy chu ky cu
    mistActive = false;
    switch (level) {
      case 1: power = 25;  duration = 10000; break;
      case 2: power = 50;  duration = 20000; break;
      case 3: power = 100; duration = 30000; break;
      default: power = 0;  duration = 0;     break;
    }
    if (level != 0) {
      mistDuration = duration;
      mistStart = millis();
      mistActive = true;
      setMistPower(power);
    }
    lastLevel = level;
  }
}

if (mistActive) {                    // dem nguoc khong blocking
  unsigned long elapsed = millis() - mistStart;
  if (elapsed >= mistDuration) { setMistPower(0); mistActive = false; }
  else { /* in so giay con lai khi thay doi */ }
}
vTaskDelay(pdMS_TO_TICKS(50));
```

Task chờ độ ẩm từ dataAlert tối đa 50 ms. Khi có dữ liệu, nếu mức độ ẩm khác mức trước thì hủy chu kỳ cũ, chọn công suất và thời gian theo switch, ghi lại thời điểm bắt đầu bằng millis() và bật máy phun. Phần đếm ngược chạy ở mỗi vòng lặp: so sánh thời gian đã trôi với thời gian yêu cầu và tắt máy khi đủ, đồng thời in số giây còn lại ra Serial. Cách này không dùng delay() dài nên Task vẫn nhận được dữ liệu mới trong lúc đang phun.

3.6.6. TaskMQTT – kết nối và truyền dữ liệu (Core 0)

```
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);

while (true) {
  if (WiFi.status() != WL_CONNECTED) { /* thu lai sau 5 giay */ continue; }
  if (!mqtt.connected()) {
    if (mqtt.connect(clientId.c_str())) {
      mqtt.subscribe(TOPIC_PUMP);
      mqtt.subscribe(TOPIC_FAN);
    }
    /* thu lai sau 5 giay neu that bai */ continue;
  }
```

```
  mqtt.loop();
  if (xQueueReceive(dataQueue, &data, 0) == pdPASS) {
    snprintf(payload, sizeof(payload),
             "{\"temperature\":%.1f,\"humidity\":%.1f}",
             data.temperature, data.humidity);
    mqtt.publish(TOPIC_SENSOR, payload);
  }
  vTaskDelay(pdMS_TO_TICKS(20));
}
```

Task thực hiện theo đúng thứ tự đề yêu cầu: kết nối Wi-Fi trước, rồi kết nối MQTT và subscribe hai topic điều khiển, sau cùng mới đọc dataQueue để publish. Nếu mất Wi-Fi hoặc MQTT, Task tự thử kết nối lại mỗi 5 giây theo cách không chặn. mqtt.loop() được gọi liên tục để duy trì kết nối và nhận lệnh. Chuỗi JSON được tạo bằng snprintf() rồi publish lên lab3/de2/sensor. Mọi thao tác MQTT chỉ nằm trong TaskMQTT nên không có xung đột giữa các Task.

3.6.7. Hàm setup() và loop()

```
dataQueue = xQueueCreate(1, sizeof(SensorData));
dataAlert = xQueueCreate(1, sizeof(float));

xTaskCreatePinnedToCore(TaskMQTT,     "Task MQTT",  6144, NULL, 1, NULL,
0);
xTaskCreatePinnedToCore(TaskReadData, "Task Read",  4096, NULL, 2, NULL, 1);
xTaskCreatePinnedToCore(TaskAlert,    "Task Alert", 4096, NULL, 2, NULL, 1);

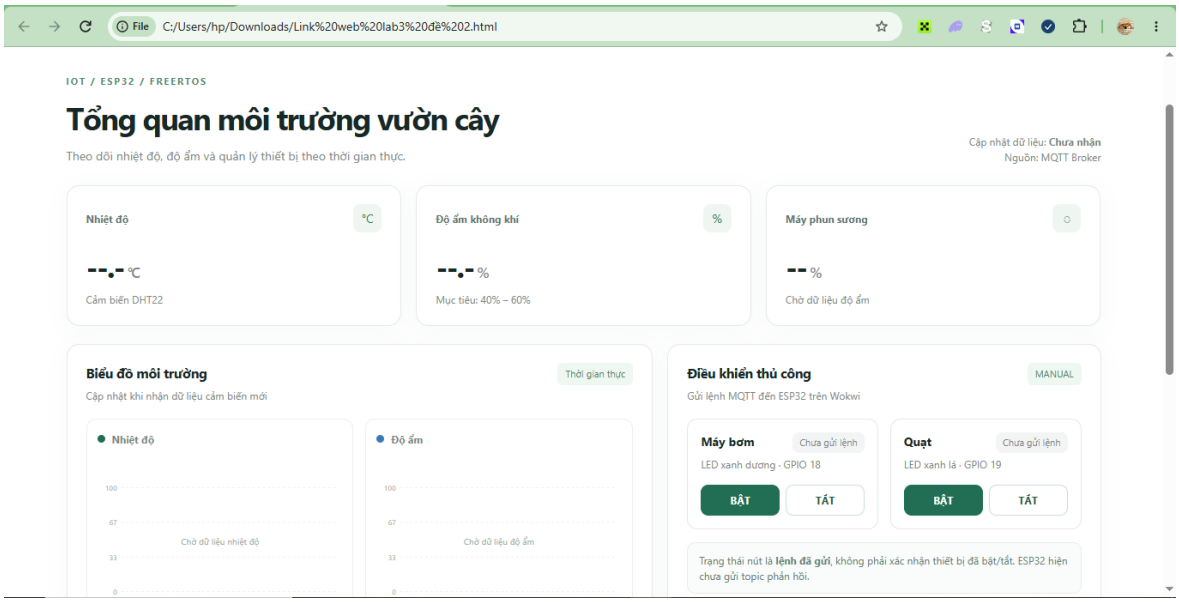
void loop() { vTaskDelay(pdMS_TO_TICKS(1000)); }
```

setup() cấu hình chân, khởi động DHT22, tạo hai Queue rồi tạo ba Task bằng xTaskCreatePinnedToCore(); tham số cuối là số Core (0 hoặc 1), tham số ngay trước là mức ưu tiên. Nếu tạo Queue hoặc Task thất bại, chương trình báo lỗi qua Serial. loop() không làm gì ngoài việc nhường CPU vì toàn bộ công việc đã nằm trong các Task.

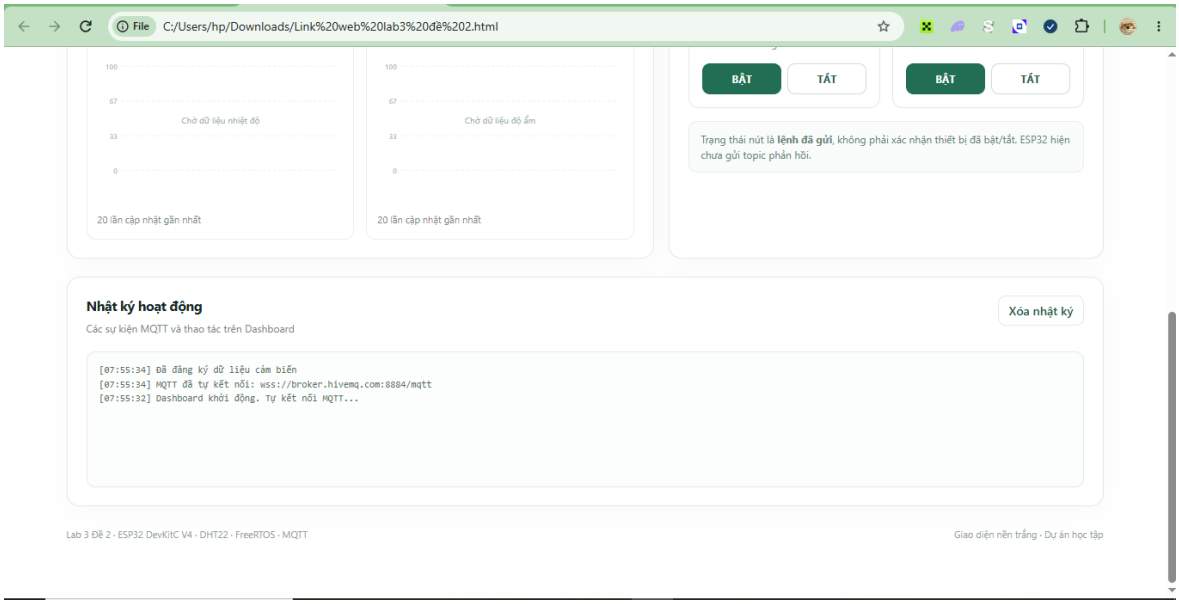
4. KẾT QUẢ ĐẠT ĐƯỢC

Các hình dưới đây chụp từ quá trình chạy mô phỏng Wokwi cùng Web Dashboard. Link project Wokwi: **https://wokwi.com/projects/479992109740419073**

4.1. Giao diện Web Dashboard



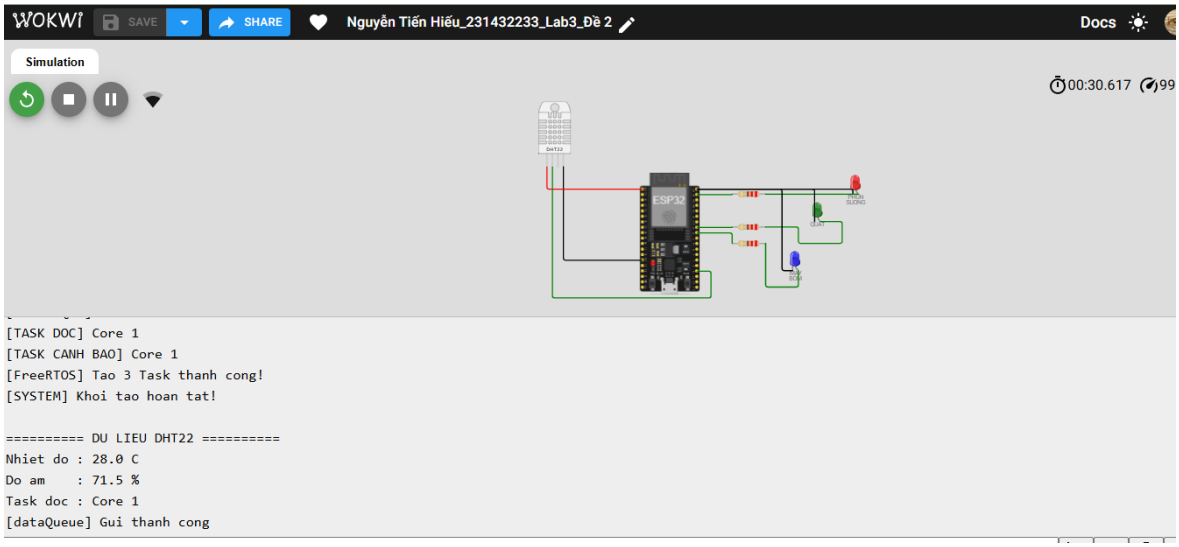
Hình 4. *Giao diện Web Dashboard: thẻ số, biểu đồ và điều khiển thủ công bơm/quạt*



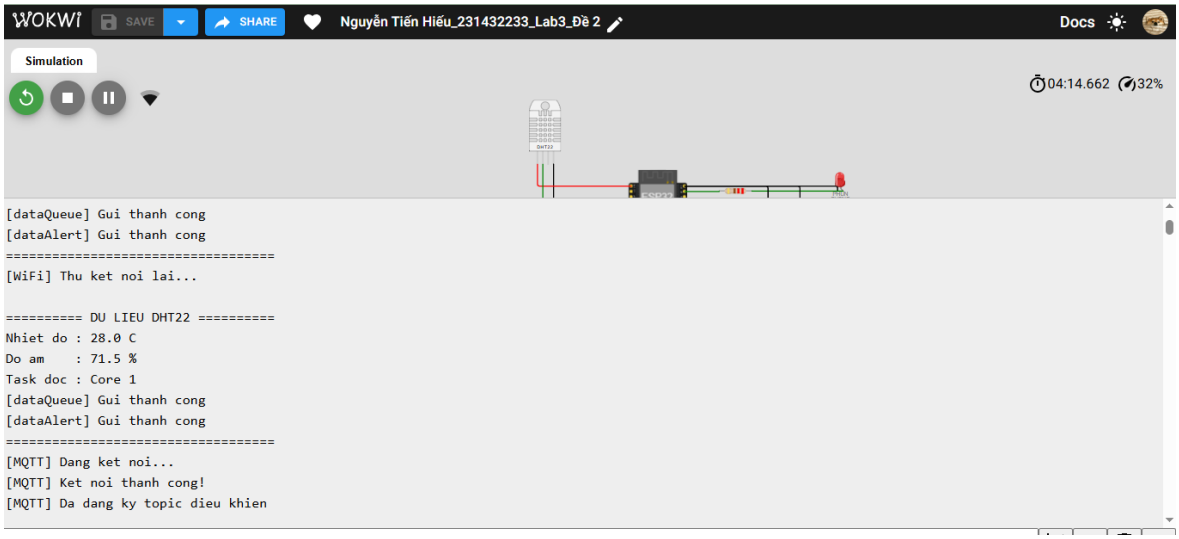
Hình 5. *Giao diện Web hiển thị nhật ký hoạt động*

4.2. Khởi động hệ thống và kết nối

Khi chạy, Serial in ra thông báo tạo Queue và ba Task thành công, mỗi Task báo Core đang chạy (Task đọc và Task cảnh báo ở Core 1, Task MQTT ở Core 0), sau đó in dữ liệu DHT22 và xác nhận đã gửi vào hai Queue (Hình 6). Hình 7 cho thấy Wi-Fi và MQTT kết nối thành công.



Hình 6. *Serial Monitor hiển thị nhiệt độ, độ ẩm và ba Task FreeRTOS*



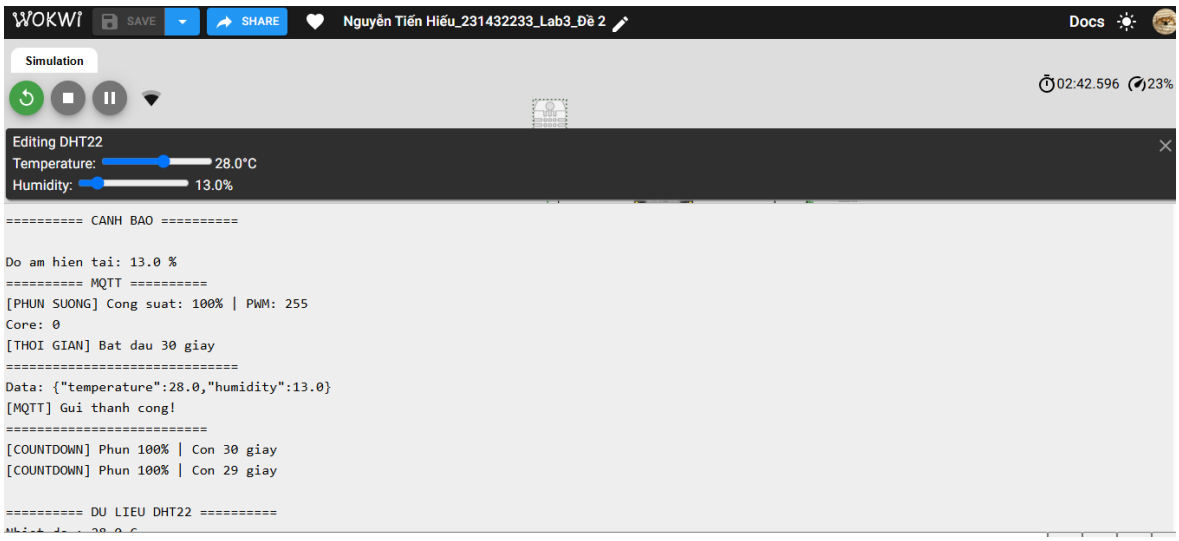
Hình 7. *Serial hiển thị kết nối Wi-Fi và MQTT thành công*

4.3. Hoạt động của máy phun sương theo độ ẩm

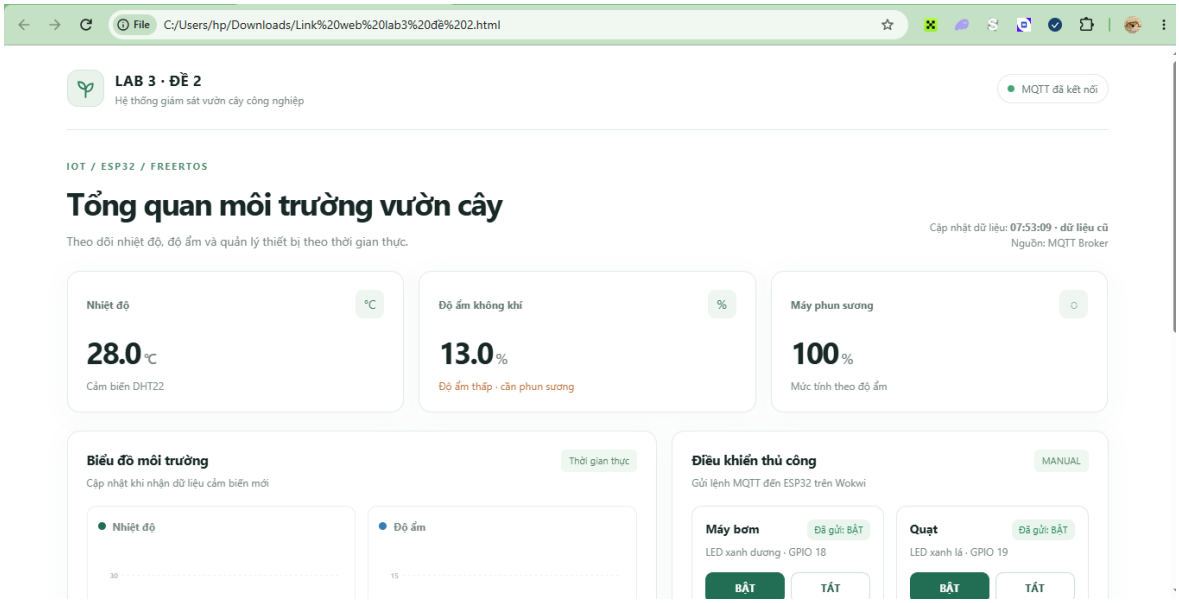
Độ ẩm trên DHT22 được thay đổi trực tiếp trong Wokwi để kiểm tra từng dải. Kết quả đối chiếu ở Bảng 10 và các Hình 8 – 15, mỗi dải có một ảnh Serial và một ảnh Web.

Bảng 10. Kết quả kiểm tra theo dải độ ẩm

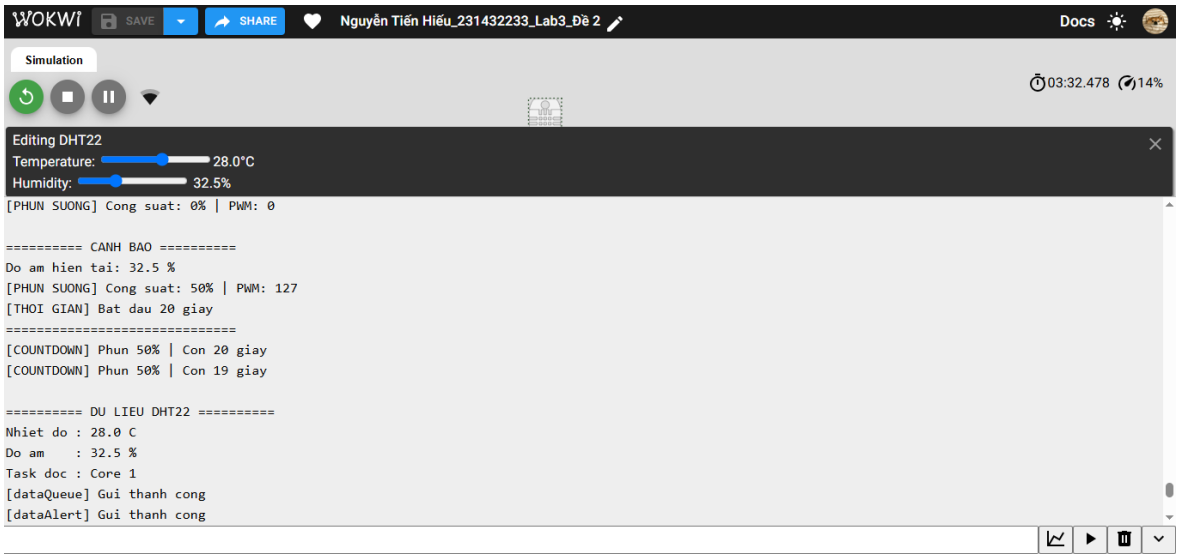
Dải độ ẩm	Công suất	PWM	Thời gian	Hình minh họa
H < 30%	100%	255	30 giây	Hình 8, 9
30% ≤ H < 35%	50%	127	20 giây	Hình 10, 11
35% ≤ H < 40%	25%	63	10 giây	Hình 12, 13
H ≥ 40%	0% (tắt)	0	0 giây	Hình 14, 15



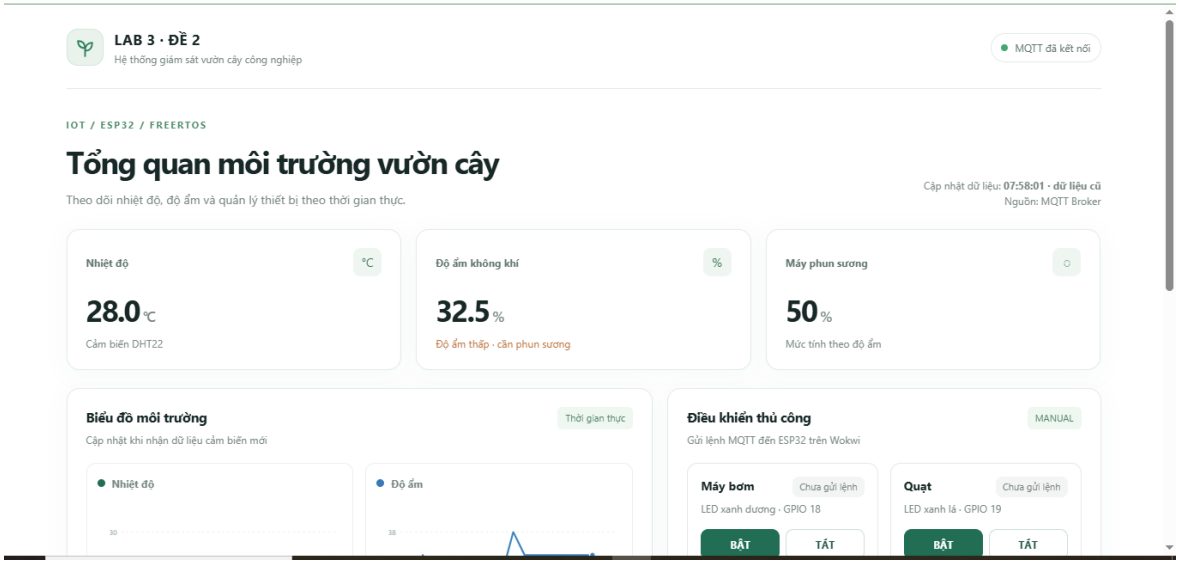
Hình 8. *Máy phun sương hoạt động ở mức 100% trong 30 giây (Serial)*



Hình 9. *Máy phun sương hoạt động ở mức 100% trong 30 giây (Web)*



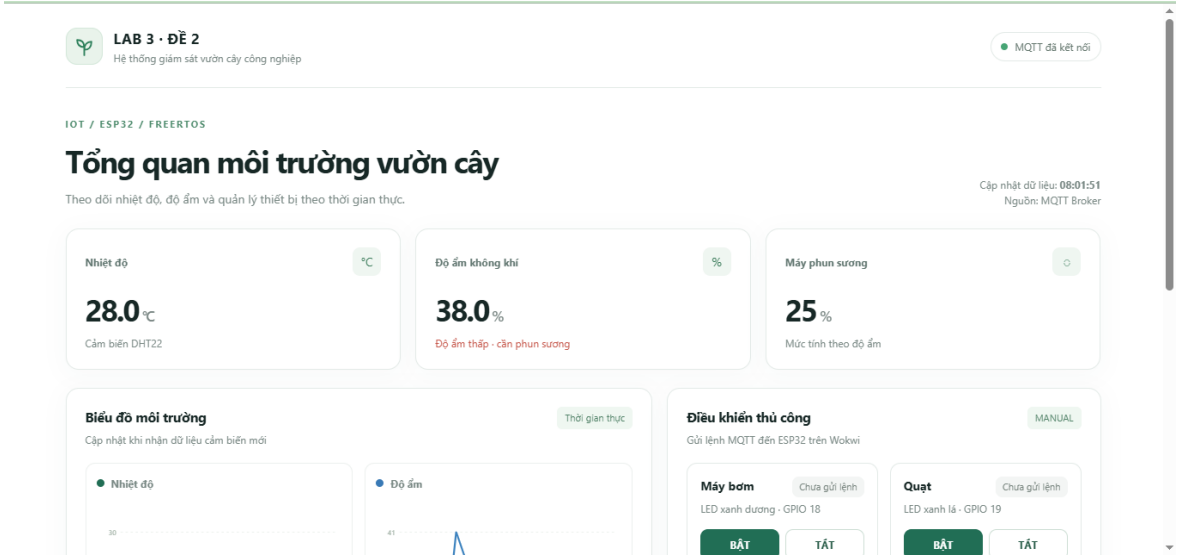
Hình 10. *Máy phun sương hoạt động ở mức 50% trong 20 giây (Serial)*



Hình 11. *Máy phun sương hoạt động ở mức 50% trong 20 giây (Web)*



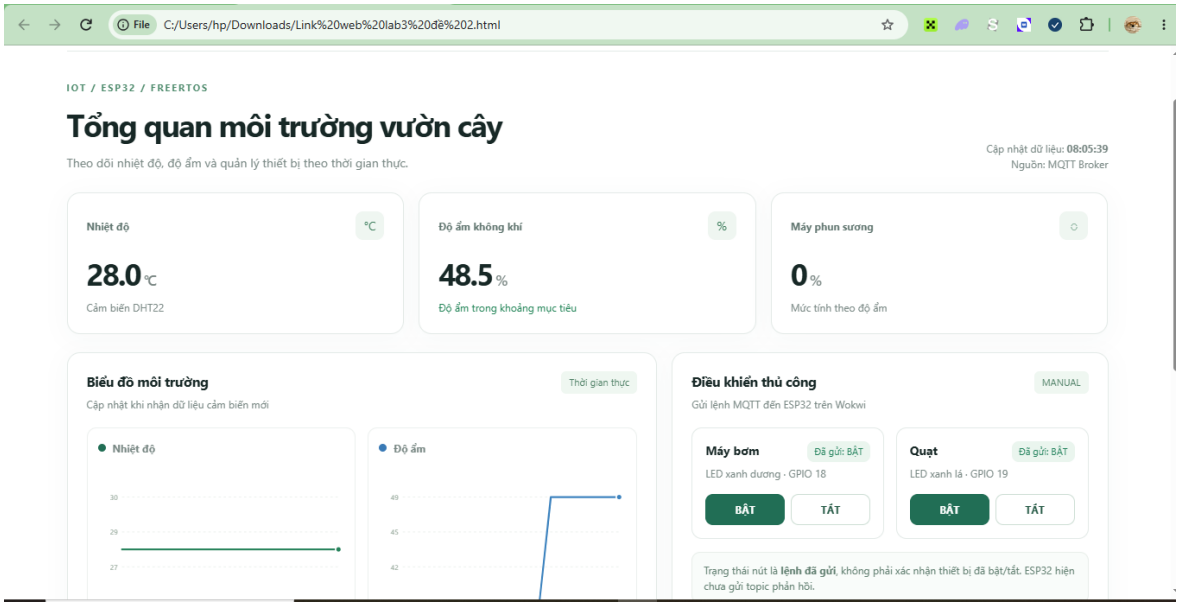
Hình 12. *Máy phun sương hoạt động ở mức 25% trong 10 giây (Serial)*



Hình 13. *Máy phun sương hoạt động ở mức 25% trong 10 giây (Web)*



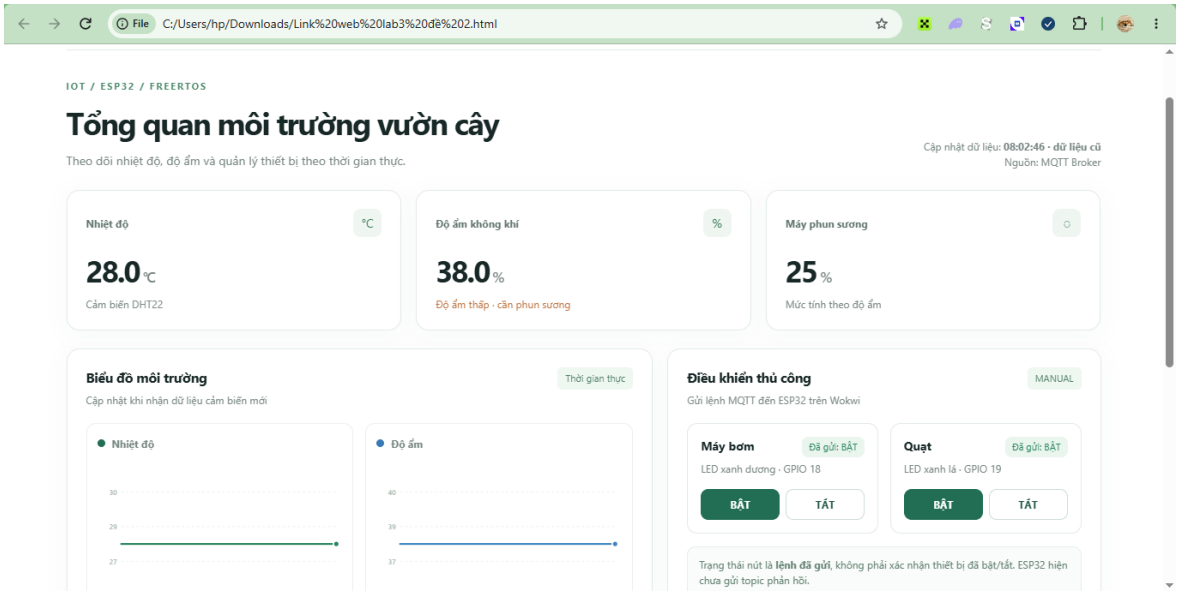
Hình 14. *Máy phun sương tắt ở mức 0% (Serial)*



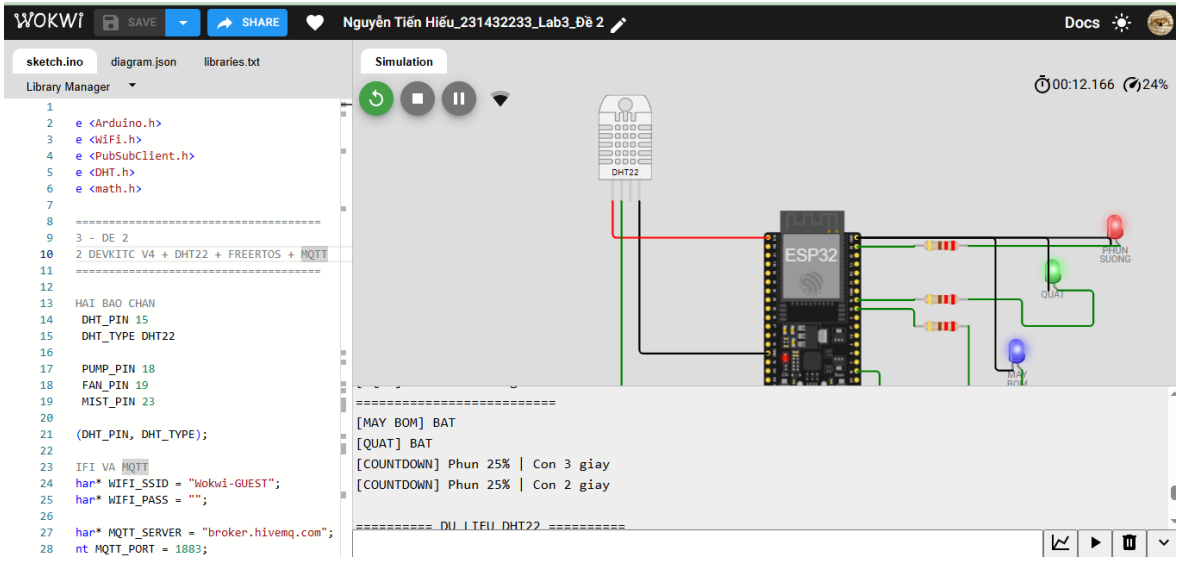
Hình 15. *Máy phun sương tắt ở mức 0% (Web)*

4.4. Điều khiển thủ công máy bơm và quạt từ Web

Khi bấm BẬT/TẮT trên Web, lệnh được publish lên topic tương ứng; ESP32 nhận lệnh qua mqttCallback() và in trạng thái ra Serial (Hình 16 và 17).



Hình 16. *Điều khiển máy bơm và quạt từ Web*



Hình 17. *Serial hiển thị lệnh bật/tắt nhận được từ Web*

5. NHẬN XÉT, HẠN CHẾ VÀ KẾT LUẬN

5.1. Nhận xét

- Hệ thống đáp ứng đủ các yêu cầu của Đề 2: đọc DHT22, hiển thị Serial, gửi MQTT, Web có thẻ số, biểu đồ và nút điều khiển, ba Task đúng Core và hai Queue đúng vai trò.
- Máy phun sương hoạt động đúng ba mức 25% / 50% / 100% với thời gian 10 / 20 / 30 giây và tự tắt khi hết thời gian hoặc khi độ ẩm trở về từ 40% trở lên.
- Việc đếm thời gian bằng millis() giúp Task cảnh báo không bị chặn; phân tách Task giúp việc mất kết nối Wi-Fi không làm dừng đọc cảm biến hay điều khiển máy phun.

5.2. Hạn chế và hướng phát triển

- Đề không quy định việc **phun lặp**; chương trình hiện chỉ bắt đầu chu kỳ khi đổi dải độ ẩm, nên nếu độ ẩm ở yên trong một dải thấp thì sẽ không phun lại sau khi hết thời gian. Có thể bổ sung chu kỳ phun lặp có thời gian nghỉ.
- Trạng thái nút trên Web là **lệnh đã gửi**, không phải xác nhận thiết bị đã chạy, vì ESP32 chưa gửi topic phản hồi. Có thể thêm topic trạng thái (ví dụ lab3/de2/status) để Web hiển thị trạng thái thực.
- Mức phun sương trên Web được **tính từ độ ẩm** theo cùng thuật toán chứ không đọc trực tiếp từ ESP32; nên bổ sung publish mức công suất thực tế.
- Khi độ ẩm vượt 60% hiện mới chỉ có cảnh báo trên Web; có thể bật quạt tự động trong trường hợp này.
- Broker công cộng không có xác thực; khi triển khai thực tế nên dùng Broker riêng, tên topic riêng và bật TLS/tài khoản đăng nhập.

5.3. Kết luận

Bài Lab đã xây dựng thành công hệ thống giám sát và điều khiển môi trường vườn cây công nghiệp trên ESP32 với FreeRTOS: ba Task chạy trên hai Core, giao tiếp bằng hai Queue, truyền dữ liệu qua MQTT và hiển thị, điều khiển từ Web Dashboard. Qua bài Lab, người thực hiện nắm được cách tổ chức Task, ghim Core, dùng Queue để trao đổi dữ liệu, xử lý thời gian không chặn và kết nối MQTT hai chiều giữa thiết bị với giao diện web.